

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Ígor Rodrigues Rueda

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 05/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	7
4.4	Atividade 4 — Controle de acesso por papel — RBAC	15
4.5	Atividade 5 — Logs e auditoria	17
4.6	Atividade 6 — Upload e perfil de usuário	19
5	Atividades extras (opcional)	21
5.1	Atividade E2 — CI/CD com GitHub Actions	21
6	Considerações finais	22
7	Declaração de autoria	23

1 Introdução

Este relatório apresenta as atividades desenvolvidas ao longo da disciplina de introdução à computação em nuvem, que visavam, por meio da implementação de microsserviços, demonstrar o funcionamento das arquiteturas utilizadas em sistemas de nuvem. O fio condutor prático do bimestre foi a construção de um sistema completo de catálogo de filmes integrado em tempo real a API do The Movie DataBase (TMDb), estruturado com foco em boas práticas de engenharia de software, isolamento de responsabilidades e controle de acesso.

Para atender a esses objetivos, a aplicação foi arquitetada em quatro microsserviços independentes (catálogo público, serviço de autenticação, serviço de log de auditoria via Redis Streams e serviço de perfil/upload de fotos integrado a um Object Storage S3-compatível), comunicando-se de forma segura e restrita através de uma rede interna em Docker. O projeto implementa o modelo de controle de acesso baseado em papéis (RBAC) com cinco níveis hierárquicos, garantindo que restrições de segurança sejam sempre validadas no backend por meio de tokens JWT em cookies httpOnly, além de contar com rastreabilidade completa de auditoria e testes automatizados via pipeline de CI/CD.

O documento está organizado de forma a detalhar as decisões arquiteturais tomadas em cada etapa do projeto. Nas seções seguintes, são descritos o padrão de comunicação e as estratégias de isolamento entre os microsserviços, a modelagem e o gerenciamento de permissões de usuários, os fluxos de segurança e auditoria de ações, bem como os procedimentos de infraestrutura e implantação em ambiente de nuvem.

2 Metodologia

Todas as atividades foram desenvolvidas no mesmo repositório público do GitHub (<https://github.com/Igor-RR/API-Tom-Hanks>), como continuação de um único projeto (catálogo de filmes - Tom Hanks). A data e a hora de cada entrega foram extraídas do histórico de commits (git log) e conferidas na página do commit no GitHub. Os prints mostram as issues com a menção ao professor, o histórico de commits, os fluxos de automação e o sistema em execução.

O projeto foi concebido sob uma arquitetura de quatro microsserviços independentes (catálogo, serviço de autenticação, serviço de logs e serviço de perfil), orquestrados através do Docker/Docker Compose (localmente) e Portainer (hospedado em nuvem). O backend foi construído em Node.js utilizando o framework Express, enquanto a interface de usuário foi desenvolvida com HTML, CSS e JavaScript puros. Os dados cinematográficos exibidos pelo catálogo são consumidos em tempo real a partir da API externa do TMDb. A camada de persistência e armazenamento foi fragmentada conforme a natureza dos dados: utilizou-se o MariaDB remoto para os dados estruturados e relacionais (usuários, favoritos, comentários e tier lists); o Redis, operando com a estrutura de Streams, foi designado exclusivamente para gerenciar o alto volume de gravações do log de auditoria de forma assíncrona; e o Garage, um object storage compatível com S3, foi implementado para hospedar isoladamente os arquivos das fotos de perfil.

Para garantir a qualidade e automatizar as entregas, o ciclo de vida do software foi gerido por uma esteira de integração e entrega contínua (CI/CD) via GitHub Actions. Este pipeline foi configurado para executar testes automatizados contra um banco de dados MariaDB efêmero e, após validação, publicar as imagens versionadas no Docker Hub. O deploy final e a gestão dos contêineres em produção foram realizados através da plataforma Portainer.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	03/10/2026 15:47	entregue com atraso
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	16/08/2026 17:23	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	23/08/2026 19:38	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	30/08/2026 15:05	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	13/09/2026 17:30	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	03/10/2026 22:49	entregue com atraso
E2	CI/CD com GitHub Actions	Pipeline que testa e faz deploy a cada push.	sem prazo	18/09/2026 17:05	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	03/10/2026 15:47
Situação	entregue com atraso
Evidência	Realizada em sala — print do sistema rodando localmente (e do repositório, se foi versionado)
Link	https://github.com/Igor-RR/agenda-telefonica-flask/commit/a1409c922834fc3831584593e49795ccef88265

O que foi feito.

Antes de descrever o que foi feito, uma ressalva: Esta atividade foi iniciada em sala e finalizada na data planejada (07/08/2026), mas não foi versionada no GitHub: entendi, na época, que se tratava apenas de

uma demonstração , consequentemente apaguei o projeto do computador. Para não permitir que esta atividade conste como não entregue no quadro de entregas, refiz-a e versionei, implicando na data de entrega registrada (03/10/2026).

A atividade em si consiste em uma aplicação que implementa uma API REST simples feita em Flask, para cadastrar contatos, sendo estes salvos em um banco de dados em json.

```
commit a1409c922834fc3831584593e49795ccef88265 (HEAD -> main, origin/main)
Author: Igor <igordriguesrueda15@gmail.com>
Date: Sat Oct 3 15:47:57 2026 -0300

Finalizando a entrega do projeto
```

Figura 1: Atividade 1 — evidência da entrega



Figura 2: Atividade 1 - resultado

Dificuldades e como foram resolvidas.

Como foi uma atividade mais simples, não tive grandes dificuldades quanto ao desenvolvimento da aplicação. A grande dificuldade foi realizar o deploy para o Portainer, foi a primeira vez que hospedei uma aplicação em um serviço de nuvem.

A solução foi continuar tentando ativamente até conseguir subir a aplicação, seguindo também as orientações do PDF do professor enviado na WhatsApp.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	16/08/2026 17:23
Situação	entregue
Evidência	GitHub — commit + README + print do catálogo

Link	https://github.com/Igor-RR/API-Tom-Hanks/commit/9fb9db92d6c7a02a881676fccfb267b4bd4bed6d
------	---

O que foi feito.

Nesta atividade, foi implementada a estrutura inicial do projeto em ambiente de nuvem. O trabalho incluiu a construção do catálogo principal consumindo dados da API externa do TMDb (direcionada aos filmes do Tom hanks), além da criação do sistema de cadastro e consulta em banco de dados remoto para a autenticação de usuários.

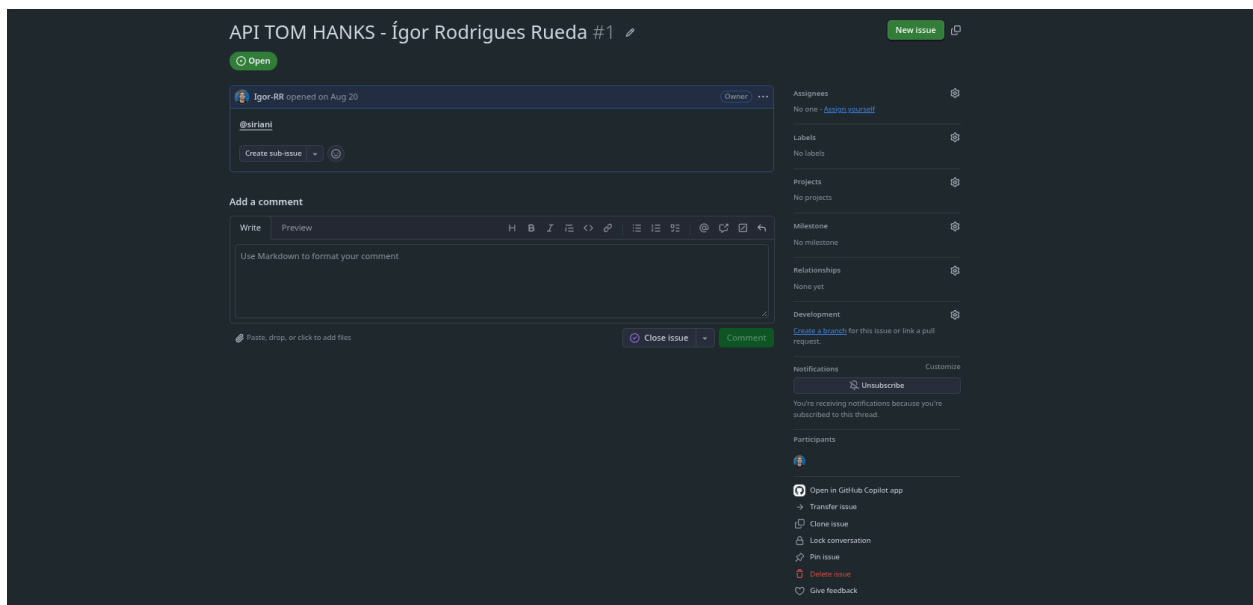


Figura 3: Atividade 2 — issue



Figura 4: Atividade 2 — GitHub - commit

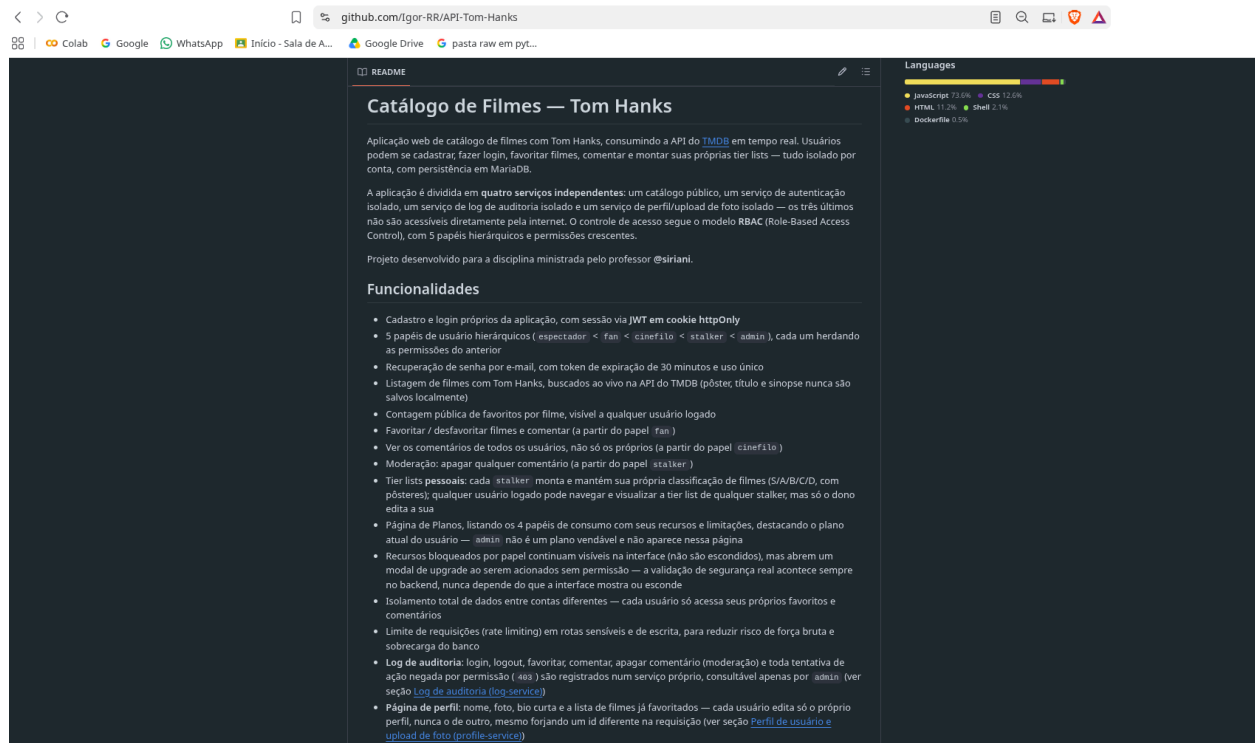


Figura 5: Atividade 2 — README

Filmes com Tom Hanks

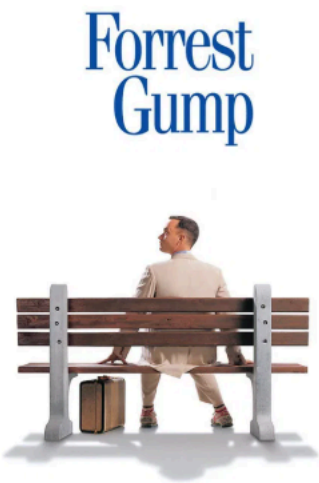
Admin

Perfil

Planos

Tier List

Sair



Forrest Gump

A man with a low IQ has accomplished great things in his life and been present during significant historic events—in each

★ Favoritar 4 favoritos

Escreva um comentário...

Comentar

Legal 

Estranho 

Olá 



The Simpsons Movie

After Homer accidentally pollutes the town's water supply, Springfield is encased in a gigantic dome by the EPA

★ Favoritado 5 favoritos

Escreva um comentário...

Comentar

Ok 

eewsw 



Saving Private Ryan

As U.S. troops storm the beaches of Normandy, three brothers lie dead on the battlefield, with a fourth trapped behind

★ Favoritado 3 favoritos

Escreva um comentário...

Comentar

Figura 6: Atividade 2 — print do catálogo (print retirado da aplicação atual, não foi solicitado na atividade, portanto não consta no README)

Dificuldades e como foram resolvidas.

As maiores dificuldades nesta atividade foram a injeção de variáveis de ambiente e deploy. Foi a primeira vez que criei uma aplicação que utilizava dados sensíveis e que não poderiam ser versionados. Da mesma forma, tive dificuldade de subir a aplicação no Portainer devido o desconhecimento desta aplicação. A solução encontrada foi a persistência, por meio de tentativa e erro foi possível subir a aplicação e uso do .gitignore para não expor as variáveis de ambiente.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026

Data realizada	23/08/2026 19:38
Situação	entregue
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando
Link	https://github.com/Igor-RR/API-Tom-Hanks/commit/7b9a9c0a96afd4be33043f8313da84fa86f4ebd8

O que foi feito.

Nesta etapa, a arquitetura da aplicação evoluiu de um modelo monolítico para um ecossistema baseado em microsserviços. Toda a lógica de autenticação — compreendendo cadastro, login, controle de papéis de acesso (roles) e redefinição de credenciais — foi desacoplada da interface do catálogo e isolada em um contêiner independente (auth-service). Como decisão estrutural e de segurança, este microsserviço não expõe portas públicas para a internet, restringindo sua comunicação exclusivamente à rede interna do Docker. Dessa forma, a aplicação principal continua sendo o único ponto de entrada acessível pelo navegador, encarregando-se de encaminhar as requisições de identidade ao serviço interno de forma invisível ao usuário final.

Além do isolamento arquitetural, implementou-se um fluxo funcional de recuperação de senha por e-mail. Para garantir a segurança desse processo, o sistema gera tokens únicos e aleatórios, armazenados no MariaDB na tabela `reset_tokens`, que possuem validação de uso único e expiração estrita de 15 minutos. Para o envio transacional das mensagens, optou-se pela integração com o servidor SMTP do Gmail, adaptando a entrega real na caixa de entrada diante de limitações de acesso a plataformas como o Brevo. Ao clicar no link, o catálogo repassa a requisição ao microsserviço de autenticação, que verifica a validade, a expiração e o status de uso do token antes de autorizar a modificação da senha.

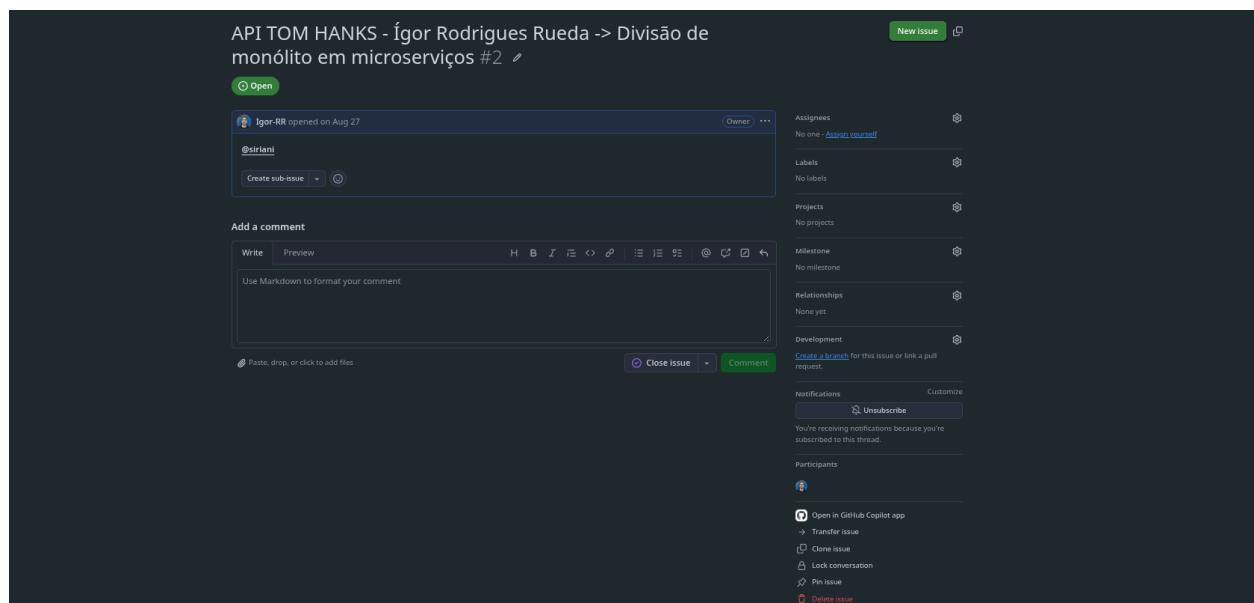


Figura 7: Atividade 3 — issue



Figura 8: Atividade 3 — evidência da entrega (commit do sistema com o Mailtrap, SMTP foi implementado posteriormente)

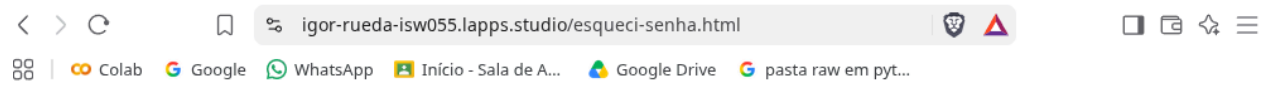


Figura 9: Atividade 3 — tela de redefinição de senha

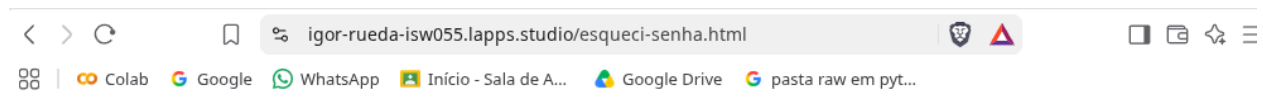


Figura 10: Atividade 3 — verificação de e-mail

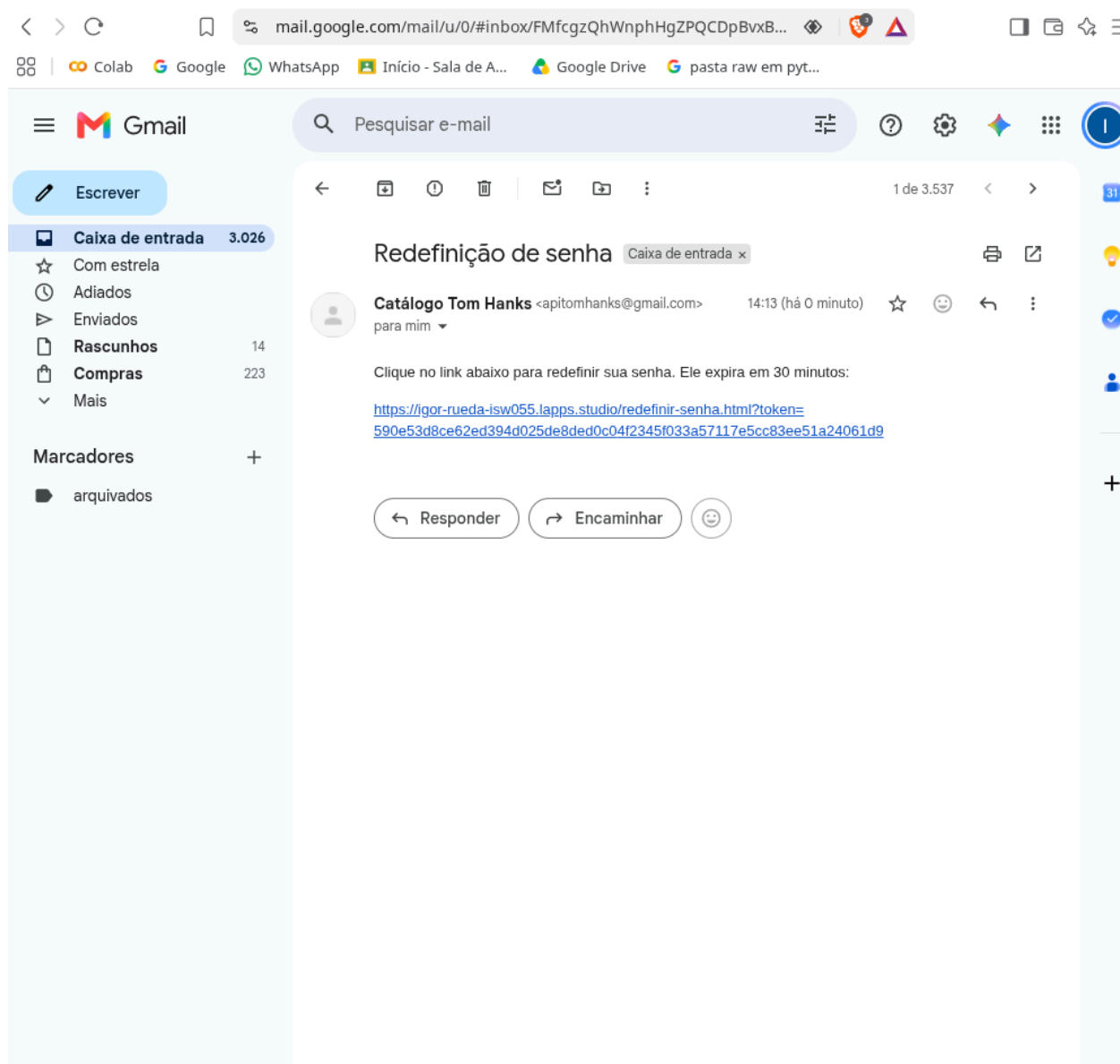


Figura 11: Atividade 3 — email de redefinição na caixa de e-mail

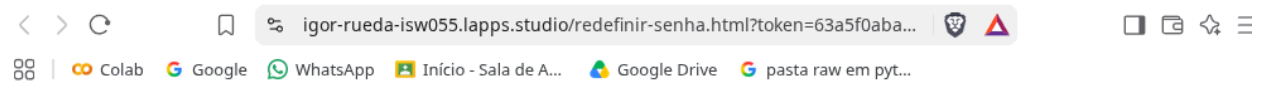


Figura 12: Atividade 3 — tela pós-token

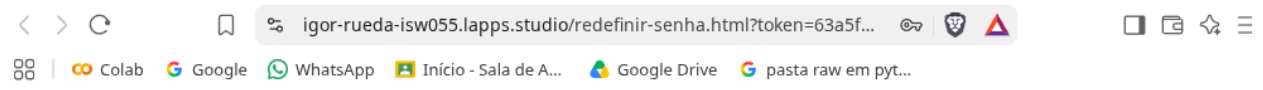


Figura 13: Atividade 3 — confirmação de recebimento de senha:

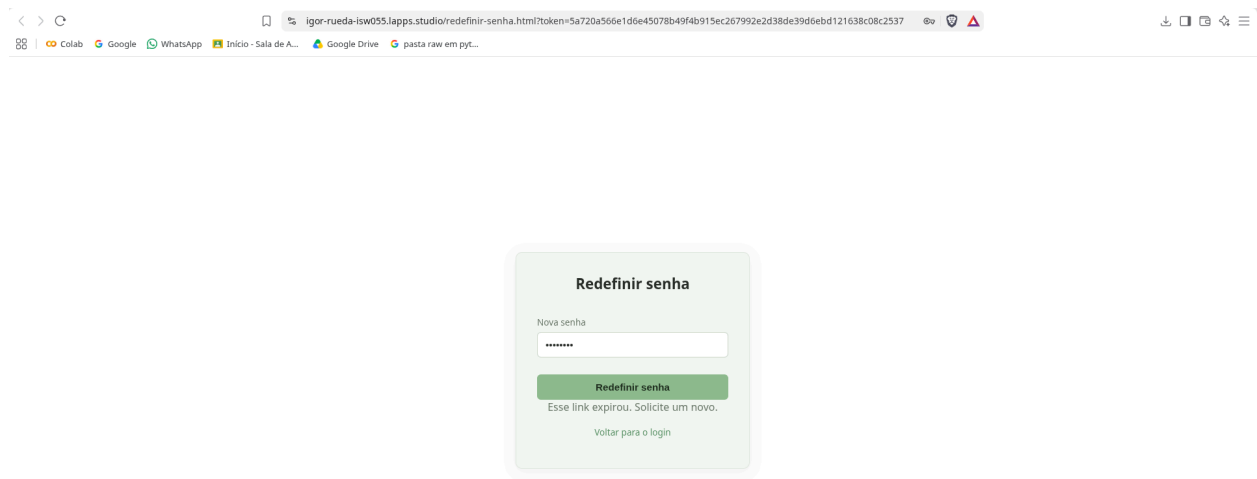


Figura 14: Atividade 3 — expiração de token



Figura 15: Atividade 3 — docker-compose.yml (catálogo + auth-service)

Dificuldades e como foram resolvidas.

A principal dificuldade nesta atividade foi a implementação do serviço de SMTP, dado que não foi possível realizar o cadastro na plataforma Brevo, pois o SMS de confirmação não chegava ao número cadastrado, impedindo a conclusão do login e acesso à API.

Diante deste cenário, optou-se por utilizar o serviço de SMTP do Gmail, o qual oferece um limite de mensagens menor (500 por dia), porém suficiente para atender os requisitos do projeto.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	30/08/2026 15:05
Situação	entregue
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/Igor-RR/API-Tom-Hanks/commit/066ff4ac3253b9d064842626eb0a859ca6083e99

O que foi feito.

Nesta etapa, foi implementado um sistema de Controle de Acesso Baseado em Papéis (RBAC) estruturado em cinco níveis hierárquicos e cumulativos: espectador, fan, cinefilo, stalker e admin. A arquitetura adotou o modelo de claims embutidas no token (Padrão B), onde a credencial de nível de acesso é assinada no JWT pelo serviço de autenticação no momento do login, com essa decisão estratégica evitando sobrecargas no banco com consultas excessivas. Integrada a este controle, foi desenvolvida a funcionalidade de tier lists pessoais, um recurso exclusivo do papel stalker persistido no MariaDB, que permite aos usuários classificar filmes em diferentes categorias hierárquicas.

Como decisão fundamental de segurança, a aplicação das regras de acesso (roles) ocorre de forma centralizada no backend, por meio do middleware `exigirNivel`. Determinadas funcionalidades são regidas por essas regras, como a tier list, comentários e favoritos, com o sistema verificando, por meio de um identificador extraído do JWT, o nível de usuário e permite o acesso a determinada funcionalidade. Isto impede que um utilizador altere a lista de outro, uma vez que o backend ignora qualquer identificador forjado no corpo da requisição. Na interface, botões restritos servem como gatilhos para modais de upgrade de plano, mas qualquer tentativa de contornar a interface acedendo diretamente a endpoints com um papel insuficiente resulta num bloqueio (HTTP 403). Os papeis de usuário e suas funcionalidades são melhor descritos na imagem abaixo:

Papel	Nível	Pode fazer
espectador	1	Ver a lista de filmes; ver a contagem de favoritos por filme; visualizar a tier list de qualquer stalker
fan	2	Tudo do espectador + favoritar/desfavoritar filmes; comentar; apagar os próprios comentários; ver apenas os próprios comentários em cada filme
cinefilo	3	Tudo do fan + ver os comentários de todos os usuários em cada filme
stalker	4	Tudo do cinéfilo + apagar qualquer comentário (moderação); criar e editar a própria tier list
admin	5	Tudo do stalker + consultar o log de auditoria

Figura 16: Atividade 4 — Tabela de roles

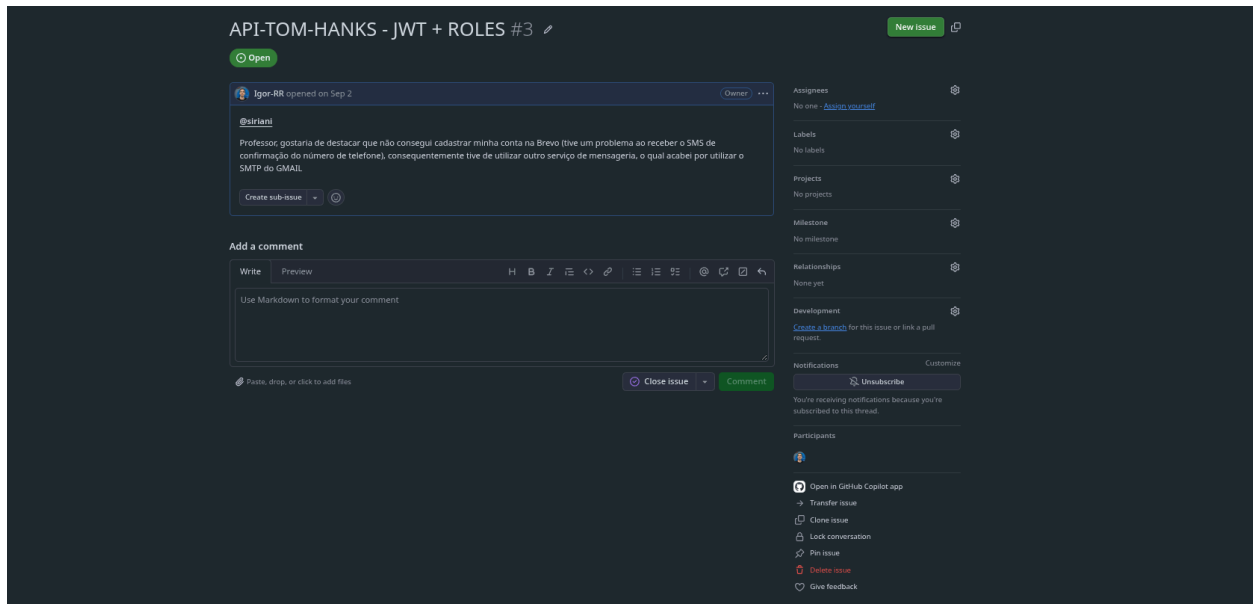


Figura 17: Atividade 4 — issue

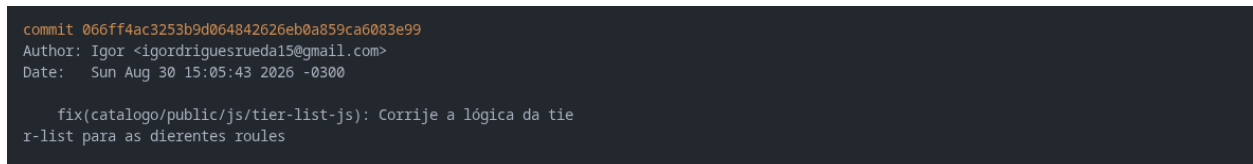


Figura 18: Atividade 4 — evidência da entrega

Tier List de igor

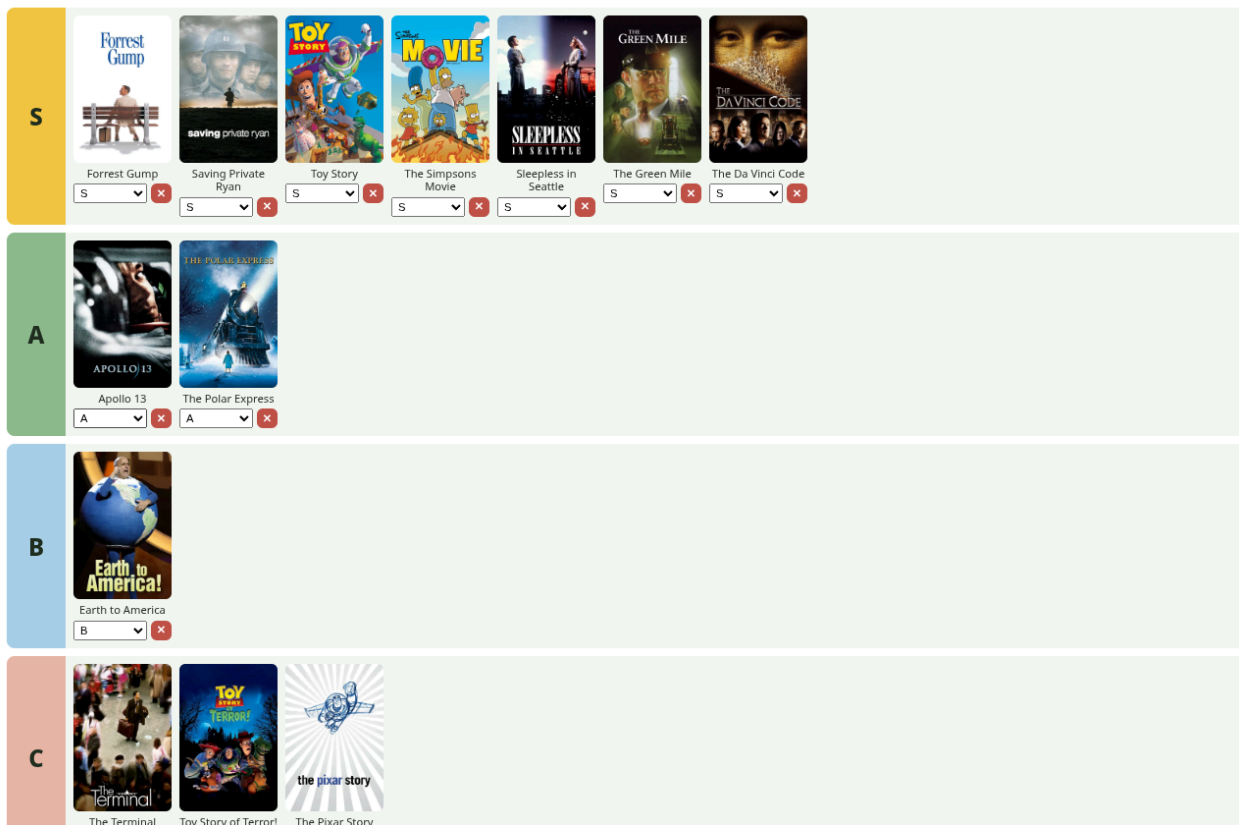


Figura 19: Atividade 4 — resultado

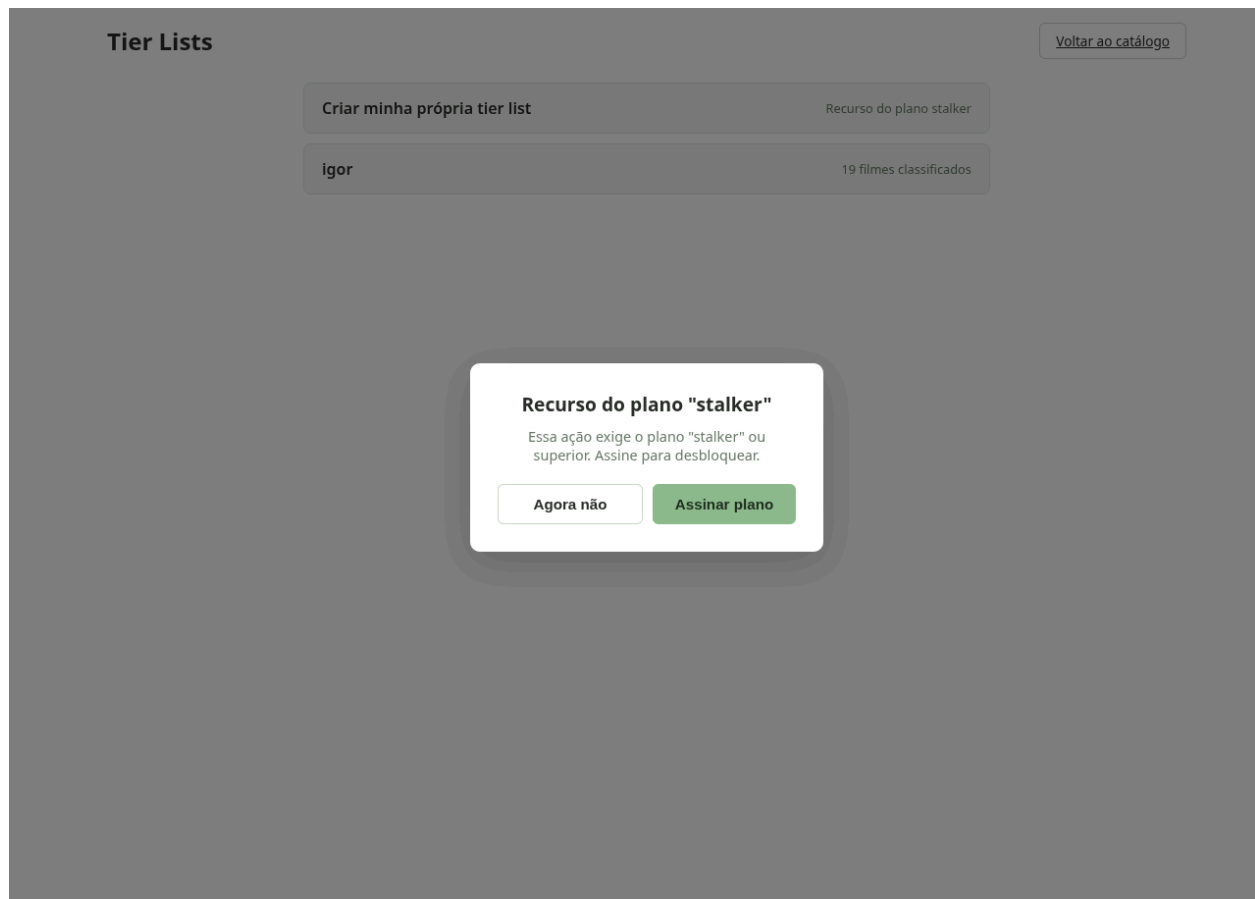


Figura 20: Atividade 4 — Recurso bloqueado pela role



Figura 21: Atividade 4 — teste 403

Dificuldades e como foram resolvidas.

Nesta atividade, a principal dificuldade foi a configuração da tierlist, que apresentou bugs relacionados ao frontend, no qual o botão de classificação não se escondia após a classificação do card de filme.

A dificuldade foi solucionada por meio de ajustes nas regras do CSS, evitando a sobreposição das configurações de alinhamento.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	13/09/2026 17:30
Situação	entregue
Evidência	GitHub — commit + print da consulta de logs pelo admin

Link	https://github.com/Igor-RR/API-Tom-Hanks/commit/0f1445354177f3bcffa311758b10ed4c245251bb
-------------	---

O que foi feito.

Para o registo de eventos de auditoria, foi desenvolvido um terceiro microserviço completamente isolado (log-service), dedicado a rastrear ações do sistema, tais como autenticações, interações e tentativas de acesso negado (HTTP 403). Como decisão de arquitetura, optou-se por utilizar o Redis com a estrutura de dados Streams em vez do MariaDB, separando a responsabilidade de logs (que exige alto volume de escrita e ordenação cronológica) dos dados relacionais de negócio. A segurança dos registos foi garantida limitando o seu acesso exclusivamente ao papel admin, sendo a consulta mediada pelo catálogo através de uma rota protegida que repassa o pedido pela rede interna.

Visando a resiliência e a performance da aplicação, a comunicação com este serviço foi estruturada sob duas camadas de assincronismo. Primeiramente, as chamadas de auditoria originadas nos outros serviços adotam o padrão fire-and-forget, assegurando que eventuais falhas no log nunca bloqueiem ou revertam a ação principal do usuário. Internamente no microserviço, implementou-se uma fila em memória que recebe os eventos e responde imediatamente com o código HTTP 202 (Aceito), delegando a um worker em background a gravação definitiva no Redis. Esta estratégia protege a aplicação contra lentidões temporárias da base de dados, sacrificando apenas a persistência dos eventos pendentes na memória em caso de reinicialização inesperada do contentor.

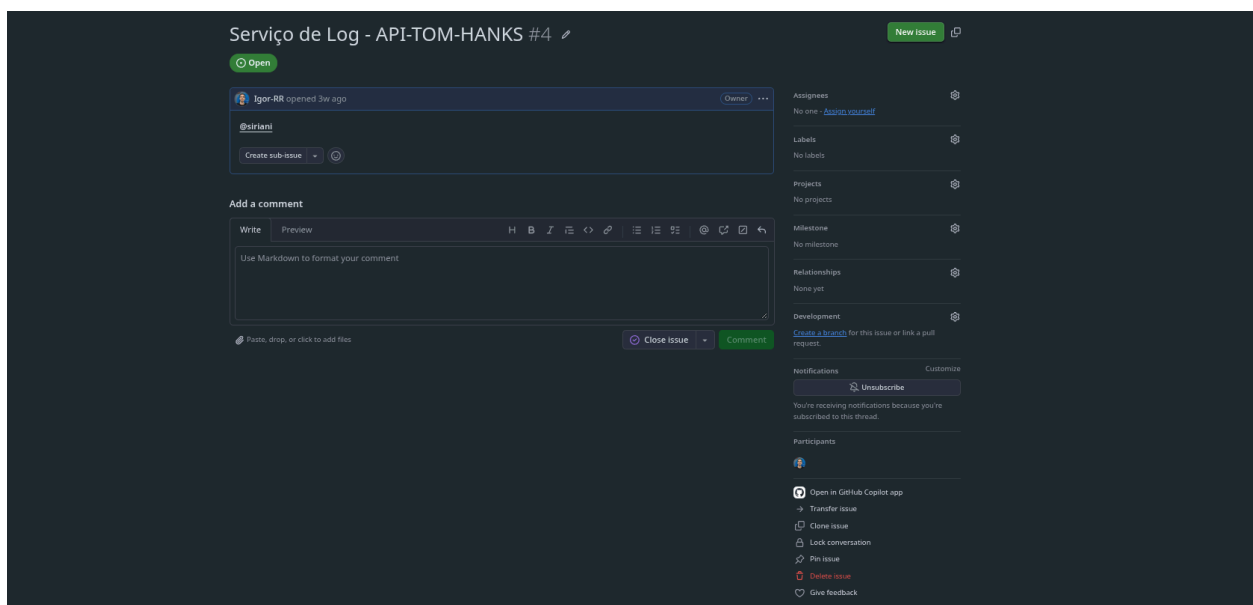


Figura 22: Atividade 5 — issue

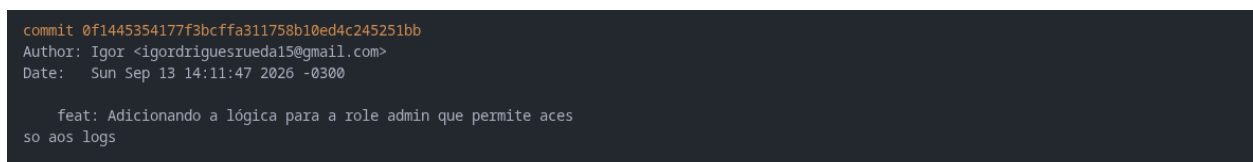


Figura 23: Atividade 5 — evidência da entrega

```

> fetch('/api/logs?limit=20')
  .then(res => res.json())
  .then(data => console.log('Logs recebidos:', data));
< ▶ Promise {<pending>}
Logs recebidos: (20) [(-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-), (-)] 4
▶ 0: {id: '1789247918389-0', usuario_id: '31', acao: 'login', timestamp: '2026-09-12T21:18:38.343Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 1: {id: '1789248440176-0', usuario_id: '31', acao: 'acesso_negado', timestamp: '2026-09-12T21:27:20.133Z', ip_origem: '::ffff:172.19.0.1', ...}
▶ 2: {id: '1789317162017-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T16:32:42.001Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 3: {id: '1789317180967-0', usuario_id: '20', acao: 'logout', timestamp: '2026-09-13T16:33:00.963Z', ip_origem: '::ffff:172.19.0.1', ...}
▶ 4: {id: '1789317400247-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T16:36:40.197Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 5: {id: '1789318476190-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T16:54:36.179Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 6: {id: '1789319273581-0', usuario_id: '20', acao: 'logout', timestamp: '2026-09-13T17:07:53.569Z', ip_origem: '::ffff:172.19.0.1', ...}
▶ 7: {id: '1789324591238-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T18:36:31.230Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 8: {id: '1789324594653-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T18:36:34.616Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 9: {id: '1789325041747-0', usuario_id: '20', acao: 'logout', timestamp: '2026-09-13T18:44:01.705Z', ip_origem: '::ffff:172.19.0.1', ...}
▶ 10: {id: '1789326895796-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T19:14:55.788Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 11: {id: '1789327055581-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T19:17:35.567Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 12: {id: '1789327921117-0', usuario_id: '', acao: 'logout', timestamp: '2026-09-13T19:32:01.100Z', ip_origem: '::ffff:172.19.0.1', ...}
▶ 13: {id: '1789328015912-0', usuario_id: '30', acao: 'login', timestamp: '2026-09-13T19:33:35.889Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 14: {id: '1789328210773-0', usuario_id: '30', acao: 'login', timestamp: '2026-09-13T19:36:50.768Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 15: {id: '1789328623997-0', usuario_id: '30', acao: 'logout', timestamp: '2026-09-13T19:43:43.649Z', ip_origem: '::ffff:172.19.0.1', ...}
▶ 16: {id: '1789328623997-0', usuario_id: '30', acao: 'login', timestamp: '2026-09-13T19:43:49.383Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 17: {id: '1789329117674-0', usuario_id: '30', acao: 'acesso_negado', timestamp: '2026-09-13T19:51:57.670Z', ip_origem: '::ffff:172.19.0.1', ...}
▶ 18: {id: '1789330114041-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T20:08:34.031Z', ip_origem: '::ffff:172.19.0.5', ...}
▶ 19: {id: '1789330115503-0', usuario_id: '20', acao: 'login', timestamp: '2026-09-13T20:08:35.490Z', ip_origem: '::ffff:172.19.0.5', ...}
length: 20
▶ [[Prototype]]: Array(0)

```

Figura 24: Atividade 5 — resultado (admin)

```

> fetch('/api/logs?limit=20')
  .then(res => console.log('Status:', res.status, res))
< ▶ Promise {<pending>}
▶ GET http://localhost:3000/api/logs?limit=20 403 (Forbidden)
Status: 403 ▶ Response {type: 'basic', url: 'http://localhost:3000/api/logs?limit=20', redirected: false, status: 403, ok: false, ...}

```

Figura 25: Atividade 5 — resultado (não admin)

Dificuldades e como foram resolvidas.

Nesta atividade não foram encontradas grandes dificuldades, apenas bugs durante a importação dos arquivos, afetando a implementação da funcionalidade de fila. Esses problemas foram corrigidos em commits consequentes, ajustando o caminho na importação de diretórios em “require()”.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	03/10/2026 22:49
Situação	entregue com atraso
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/Igor-RR/API-Tom-Hanks/commit/b76d922e58b2ca46705e503294d2f9000d135168

O que foi feito.

Nesta atividade, implementou-se o upload de foto de perfil usando o Garage, um object storage self-hosted compatível com S3 (equivalente ao MinIO sugerido no enunciado), isolado como um microserviço próprio (profile-service), seguindo o mesmo padrão de isolamento já usado no auth-service e no log-service. O arquivo da imagem vai pro Garage, e só a referência (chave do objeto) é salva no MariaDB — nunca o binário. O acesso à foto é sempre privado e autorizado pela aplicação (nunca leitura pública permanente), e cada usuário só pode editar o próprio perfil, já que o backend confere a identidade a partir do JWT da sessão, nunca de um id enviado pelo cliente.

```
docs: Atualizando README com o serviço de upload de foto de perfil

commit b76d922e58b2ca46705e503294d2f9000d135168
Author: Igor <igordriguesrueda15@gmail.com>
Date: Sat Oct 3 22:49:42 2026 -0300

fix: Corrigindo docker compose para utilização de garage como serviço da rede interna
```

Figura 26: Atividade 6 — evidência da entrega

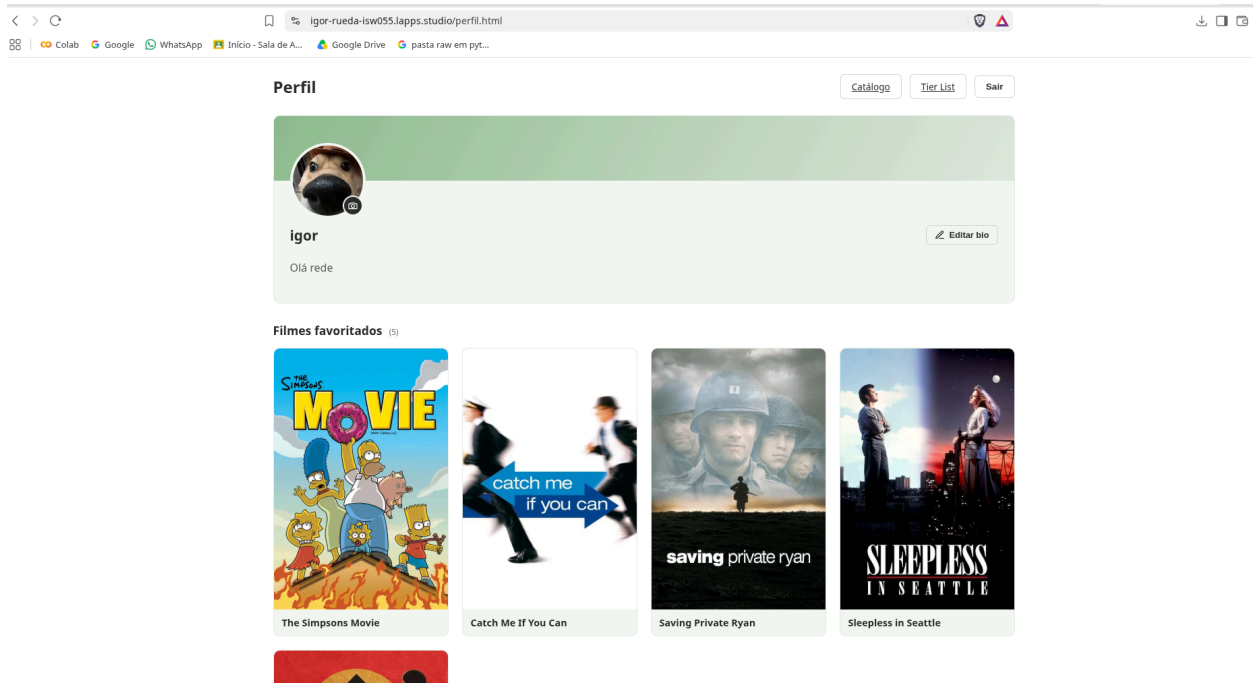


Figura 27: Atividade 6 — print do perfil com foto

A maior dificuldade desta atividade foi a tentativa de implementar o acesso direto do navegador ao Garage por URL pré-assinada — o jeito padrão de fazer isso com object storage S3-compatível. A ideia era simples: o profile-service assina uma URL temporária, publica a porta do Garage no docker-compose.yml, e o navegador baixa a imagem direto do storage, sem passar pela aplicação. Só que, depois de realizado o deploy para o Portainer, a porta extra do Garage (3900) respondia de forma inconsistente: às vezes retornava um erro rápido, às vezes a requisição ficava “pending” indefinidamente, tanto no navegador quanto via curl isolado.

Para resolver este problema, substituiu-se o acesso direto do navegador ao Garage e por um proxy autenticado através do próprio catálogo. O profile-service passou a buscar o arquivo no Garage usando as credenciais do SDK diretamente pela rede interna do Docker (sem precisar assinar URL nenhuma, já que ele mesmo tem a chave de acesso), e o catálogo expôs uma rota nova (GET /api/perfil/:usuario_id/foto) que busca esse arquivo no profile-service e repassa os bytes pro navegador, usando a mesma porta que o resto do site já usa — a única que a hospedagem garante de verdade. A decisão de segurança documentada no requisito continuou intacta (bucket privado, acesso sempre autorizado pela aplicação, nunca leitura pública permanente); só mudou quem verifica essa autorização: deixou de ser uma assinatura temporária que o navegador consumia sozinho, e passou a ser o próprio backend, a cada requisição.

5 Atividades extras (opcional)

5.1 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline que testa e faz deploy a cada push.
Data planejada	sem prazo
Data realizada	18/09/2026 17:05
Situação	entregue
Evidência	Repositório público no GitHub — commit com data e hora, README mencionando github.com/siriani, print do sistema rodando
Link	https://github.com/Igor-RR/API-Tom-Hanks/commit/bd9fca3f1f997ba40df1dd4976f8f554825325f9

O que foi feito.

Para automatizar a integração e entrega contínua, implementou-se um pipeline no GitHub Actions acionado a cada push na main. Esta esteira constrói as imagens e levanta um ambiente efêmero com MariaDB para executar testes reais de ponta a ponta, validando os fluxos de autenticação sem o uso de simulações (mocks). Após a aprovação, as imagens são publicadas automaticamente no Docker Hub com dupla etiquetagem (latest e o hash do commit), assegurando a rastreabilidade do código. O deploy final em produção, contudo, foi mantido como uma etapa deliberadamente manual através da interface do Portainer.

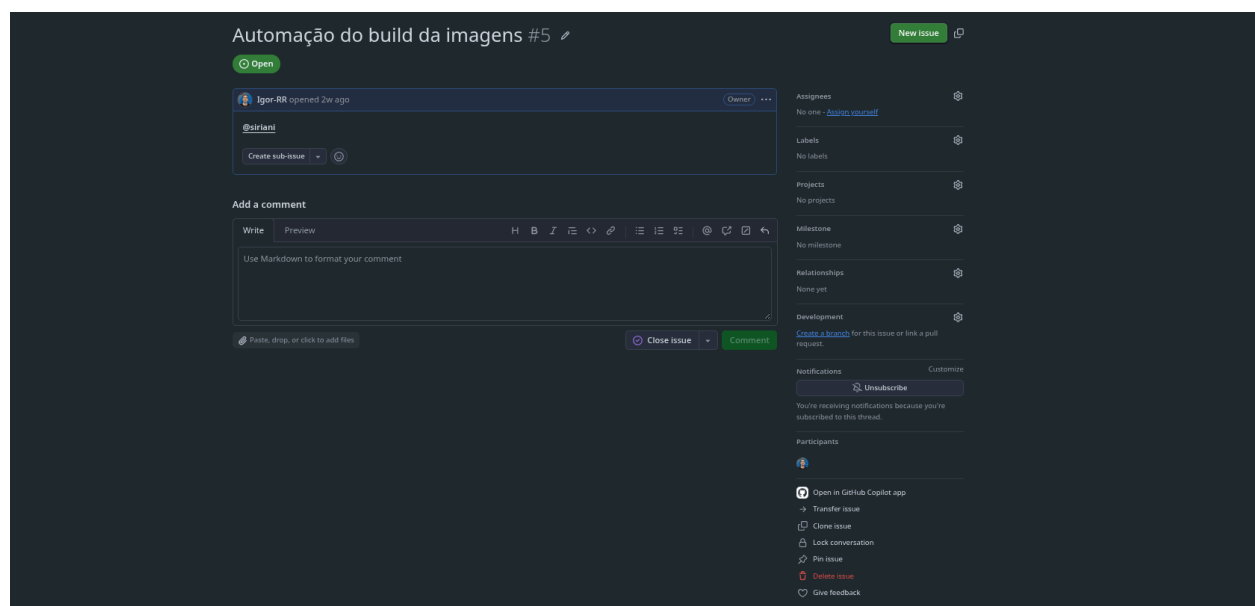


Figura 28: Atividade E2 — issue

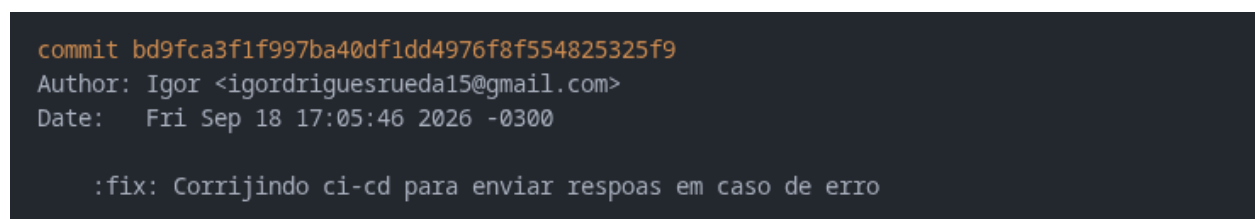


Figura 29: Atividade E2 — evidência da entrega

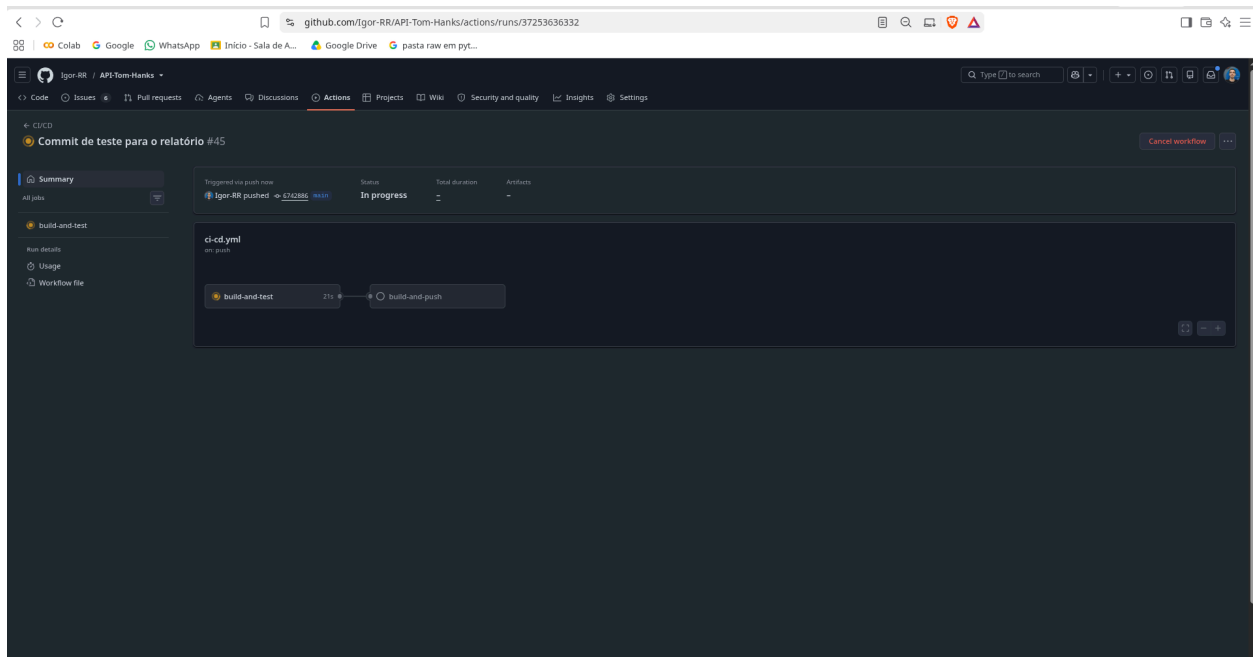


Figura 30: Atividade E2 — sistema rodando

Dificuldades e como foram resolvidas.

Nesta atividade, a maior dificuldade foi à quanto a implementação do mariaDB, que não inicializava no mesmo tempo em que era solicitado. Para corrigir este problema, foi adicionado um `start_period` de 40s.

6 Considerações finais

Esse bimestre, o maior aprendizado foi entender como funcionam as arquiteturas de nuvem, por meio de um paralelo com arquiteturas de microsserviços. Dividir a aplicação em serviços isolados (catálogo, autenticação, log de auditoria e, por último, upload de perfil) me fez enxergar de verdade por que isolar responsabilidades importa — cada serviço com seu próprio banco de acesso, sua própria rede interna, e um único ponto público pra tudo. Aprendi também bastante sobre controle de acesso de verdade: não validar permissão só na interface, sempre no backend, e nunca confiar em nada que vem do corpo da requisição — isso ficou muito claro quando implementei o requisito de “só o dono edita o próprio perfil”, comparando o id da URL com o id do token, não o que o cliente mandava.

A parte mais difícil, de longe, foi a integração entre o profile-service e o Garage em produção — não o código em si, mas a infraestrutura em volta dele. Implementei URL pré-assinada (o jeito padrão de servir arquivo de um object storage), testei localmente, parecia certo, e simplesmente não funcionava de forma confiável no servidor por causa de uma porta que a hospedagem não roteava de forma garantida. Isso me ensinou que “funciona na minha máquina” não é prova de nada, e que boa parte do trabalho real de um sistema distribuído é diagnosticar esse tipo de problema de rede/infraestrutura, não só escrever a lógica de negócio.

Pro próximo bimestre, pretendo levar comigo esse hábito de verificar restrições de infraestrutura antes de me comprometer com uma arquitetura, e também a disciplina de documentar decisões com o trade-off explícito (o que ganhei, o que perdi), que acabou me ajudando bastante a não perder o fio da meada no meio de tanta mudança. Um ponto que também ficou evidente e que quero atacar com mais cuidado é performance: hoje várias páginas fazem mais requisições do que precisariam (o catálogo, por exemplo, tinha um padrão clássico de N+1 nos comentários, um por card em vez de uma só pra todos), retardando o carregamento do perfil por de conexões de banco mal geridas.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Ígor Rodrigues Rueda

Pompeia, 05/10/2026